Experiment 10

Aim: Explore and implement Autoencoders for image denoising

Dataset Source:

MNIST Dataset Link: http://yann.lecun.com/exdb/mnist/

Dataset Description:

The MNIST dataset consists of grayscale images of handwritten digits.

Features:

- Pixel values (0–255)
- Image size: 28 × 28

Target Variable:

- No explicit target (unsupervised learning)
- Model learns to reconstruct clean images

Dataset Characteristics:

- 70,000 images
- Training: 60,000
- Testing: 10,000
- Grayscale images

Mathematical Formulation of the Algorithm:

Autoencoder consists of:

- Encoder
- Latent space
- Decoder

Encoding:

$$z = f(x) = \sigma(Wx + b)$$

Decoding:

$$\hat{x} = g(z) = \sigma(W'z + b')$$

Loss Function (Reconstruction Loss):

$$L = ||x - \hat{x}||^2$$

Where:

- xxx: original image
- x^\hat{x}x^: reconstructed image

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Algorithm Limitations:

- Requires large dataset
- May reconstruct noise if not trained properly
- Overfitting risk
- Not ideal for highly complex images
- Loss of fine details

Methodology / Workflow:

Steps:

1. Load MNIST dataset

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

2. Normalize images

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

3. Add noise to images

```
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

4. Build autoencoder:
   - Encoder
   - Bottleneck
   - Decoder

```
model = models.Sequential()

# Encoder
model.add(layers.Flatten(input_shape=(28,28)))
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(64, activation='relu'))

# Decoder
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(784, activation='sigmoid'))
model.add(layers.Reshape((28,28)))
```

5. Train model

```
model.compile(optimizer='adam', loss='mse')
```

```
model.fit(
    x_train_noisy, x_train,
    epochs=10,
    validation_data=(x_test_noisy, x_test)
)
```

6. Reconstruct images

```
denoised_images = model.predict(x_test_noisy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

7. Compare noisy vs denoised images

Workflow Diagram:

Original Image → Add Noise → Autoencoder → Reconstructed Image → Evaluation

Performance Analysis:

Metrics Used:

- Mean Squared Error (MSE)
- Visual comparison

Observations:

- Noisy images become clearer after reconstruction
- Model removes random noise effectively

Interpretation:

- Lower MSE indicates better denoising
- Visual quality confirms effectiveness

Hyperparameter Tuning

Parameters Tuned:

- Number of layers
- Neurons in bottleneck
- Learning rate
- Epochs

Impact:

- Smaller bottleneck → better compression
- More layers → better reconstruction
- More epochs → improved denoising

SDG Goal	Implementation in Experiment
SDG 3: Good Health and Well-being	Image denoising improves clarity of medical images, aiding in better diagnosis and healthcare systems.
SDG 9: Industry, Innovation and Infrastructure	Autoencoders demonstrate innovation in deep learning for image processing and intelligent systems.
SDG 4: Quality Education	Helps students understand unsupervised learning and deep learning concepts practically.